ParSec: Preventing Model Identification in Inference Serving Systems

Abstract

Inference serving systems have become increasingly integral to real-time web applications, where users can query diverse models developed by independent contributors. However, querying can inadvertently expose critical intellectual property - such as model architecture, training data, and functionality - to adversarial users through black-box attacks. This paper introduces ParSec, a novel mechanism designed to prevent model identification in inference serving systems. ParSec employs differential privacy techniques to obfuscate the routing of queries. This conceals the target model within a large zoo of models. By randomizing query assignment, ParSec ensures that individual models are indistinguishable from one another, thereby preventing model identification. This effectively mitigates downstream black-box attacks, even when adversaries utilize learning-based methods. Extensive evaluations on vision models and two image classification datasets demonstrate that ParSec achieves a balance between privacy and system utility. It provides configurable privacy while maintaining high system utility. ParSec allows practical deployment through a flexible shim layer atop real inference serving systems. To our knowledge, ParSec is the first solution that addresses the challenge of model identification in inference serving systems. Access to code [1].

1 Introduction

Model Zoo Based Inference Serving Systems. The deployment of inference serving systems [1, 3, 21, 22, 28, 62] to serve machine learning models in interactive web applications [32, 73] is witnessing a significant surge. Google uses TensorFlow Serving [1] for applications like Google Photos [43] and YouTube [20] recommendations. Amazon uses SageMaker [35] for applications like Google Search for applications like Alexa [17] and AWS Rekognition [50]. As these applications are typically user-facing, ML inference must be performed in real-time and hence queries are associated with deadlines, also known as a latency service level objective (SLO) (*e.g.*, $<$ 100ms) [77]. A query's latency SLO defines the latency budget available to the system to serve that query. Model less inference serving systems [28, 62] removes the client's burden of selecting an explicit model from the set of registered models, often called the **model zoo**.

These systems decouple applications from the models they use, allowing each to evolve independently. Furthermore, this obviates the need for making premature static model choice decisions [21, 22] and enables choosing the right-sized model *just-in-time* on a query to query basis [28, 62]. This flexibility proves important for maximizing the fraction of queries for which the latency SLO is met as dynamic deployment conditions (*e.g.*, memory/network/storage bandwidth, power consumption, battery level) and application requirements (desired accuracy and response time) fluctuate [8].

Privacy Concern for Inference Serving Systems. This proliferation of model zoo inference serving systems raises significant concerns regarding the privacy of the registered models. The models stored span a large range of model sizes and performances (measured using accuracy), each potentially trained on proprietary datasets and further fine-tuned and specialized to different runtime conditions, exposing valuable intellectual property.

Each proprietary model in the model zoo is considered highly valuable because they are expensive to obtain [66]. Along with training the final model, the architecture, training algorithm, training data, and hyper-parameters are all valuable (*e.g.*, Training GPT-3 [10] cost OpenAI more than $4 million). Qualcomm's Snapdragon 8 Gen 3 [4, 5] comes with Meta's Llama 2 [68], which has up to 70 billion parameters. This implementation could be worth millions of dollars.

The cost is further compounded by specializing the model to have predictable execution times under a certain latency budget. This leverages latency-aware neural architecture search, mixed-precision quantization, hardware-aware kernel fusion, *etc.* Thus, surprisingly, smaller models may actually hold higher value to the attacker. Model zoos can contain tens to hundreds of these vulnerable models in the cloud,

[1] ParSec Repository

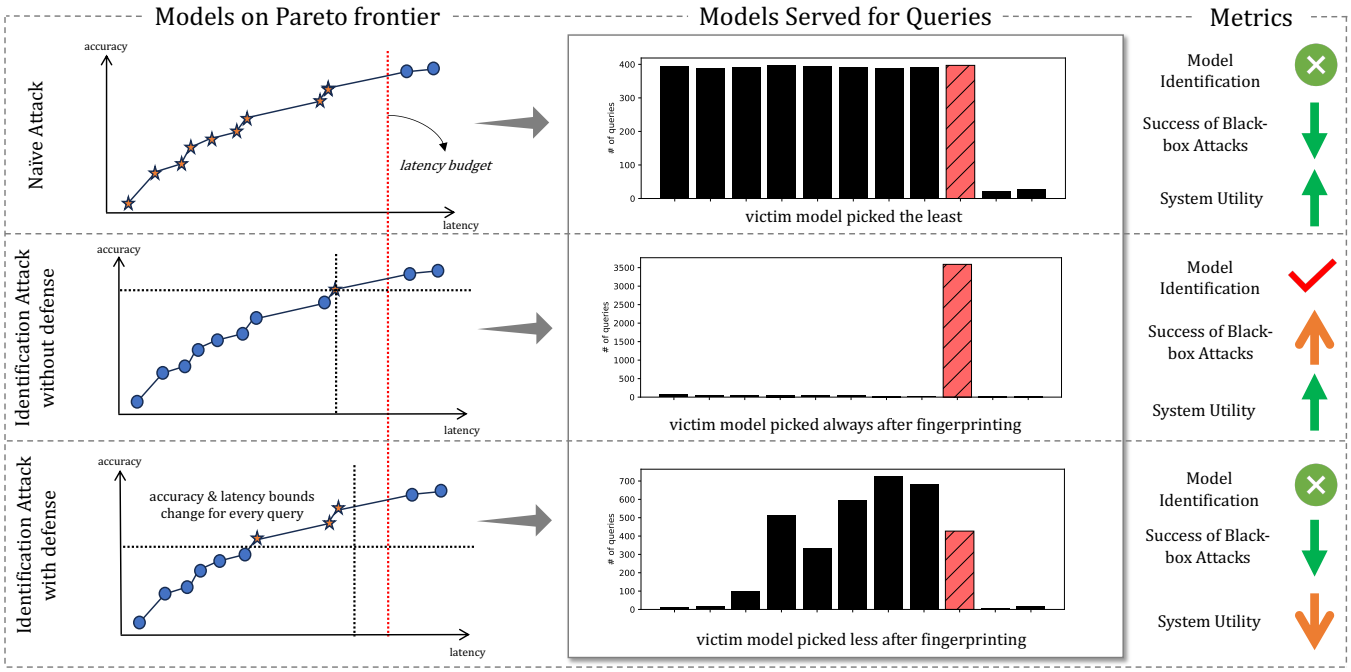


Figure 1: A high-level overview of this paper. The first Pareto frontier shows a naive attack. In the second Pareto frontier, the accuracy and latency specifications obtained from our identification algorithm successfully target the victim model, and it gets picked for serving most of the queries. In the third Pareto frontier, the accuracy and latency specifications after identification but with the defense become less accurate, and the victim model is not picked that often. Our identification algorithm shows that a single-model attack can be used on a model zoo. With our defense mechanism, we offer a protection method.

each specialized for a different application requirement (*e.g.*, Meta's anomaly detection framework, Sigma, contained more than 100 distinct models running in production everyday in 2018 [32] and Meta's personalized recommendation system contained more than 400 distinct models in 2020 [31]).

Black-Box Attacks on Inference Serving Systems. Black-box attacks [7, 27, 64, 69] pose a substantial threat to the privacy of models in inference serving systems. These attacks allow adversaries to exploit vulnerabilities within the system and extract sensitive information, including the victim model's functionality, training data, architecture, and parameters. Surprisingly, despite the increasing adoption of model-less inference serving systems [1, 21, 22, 28, 35, 62], practical demonstrations of black-box attacks on these systems have not been well studied.

Indeed, state-of-the-art model extraction techniques assume the adversary's ability to route all queries to the *same*, explicitly specified victim model. Importantly, we assert that this is an outdated and impractical assumption for state-of-the-art inference serving systems [62], which prioritize flexibility to cater to diverse user requirements. Consequently, a pressing need arises to explore effective mechanisms that can adapt existing black-box attacks, such as model extraction and membership inference, to state-of-the-art inference serving systems that are model-less, latency-aware, and operate under real-time latency constraints. To the best of our knowledge, this work is the first to show the viability of black-box attacks on inference serving systems *without requiring explicit model specification by the user*.

Our Identification Mechanism. We propose a novel and query-efficient *property* (*e.g.*, accuracy, latency, *etc.*) identification mechanism that enables black-box attacks on model-less inference serving systems. Our key technical insight is to leverage the Pareto front used for inference serving. Our identification mechanism enables black-box attacks to have scores within 1% of the corresponding values in the single-model setting while spending the same number of queries. We show up to 11% gain in accuracy and up to 10% gain in fidelity than the naive attack without identification while spending only 4000 queries. This bridges the gap and recovers the loss in performance resulting from the model-less abstraction layer.

`ParSec` Defense approach. In this paper, our key technical contribution is to propose a novel defense approach, with differential privacy (DP) [26] guarantees, to counter identification mechanisms like ours. The defense mechanism is based on the *differentially private* selection [48] of the model based on the accuracy and latency[2] specifications of queries, effectively disrupting the identification process. The mechanism

[2]Proposed attack and defense generalize to metrics other than accuracy and latency.

seeks to return the candidate model that has the maximum quality (or utility) score. To the best of our knowledge, this work is the very first rigorous application of DP in inference serving systems. We study the efficacy of our defense in preventing property identification attack, like our attack mechanism, and how it can *reduce leakage* in downstream attacks like model extraction and membership inference. Our defense strategy reduces the accuracy and fidelity of the extraction attack by up to 11% and 10%, respectively and accuracy of membership inference by up to 10%.

Finally, we expose and study a fundamental tradeoff between the effectiveness of our defense and inference serving system performance. We measure the latter in **utility** – L^1 distance from the served model's actual accuracy and latency to the input accuracy and latency. By demonstrating `ParSec` provides significant protection while maintaining $\sim 90\%$ utility, we show its *negligible overhead from DP*. Fig. 1 captures a high-level conceptual overview.

1.1 Key Contributions

We instantiate the proposed attack and defense mechanisms in our `ParSec` *shim* layer, which can be seamlessly integrated with any state-of-the-art inference serving system. Our paper instantiates the following principal contributions:

- A model-less inference serving shim layer with the implementation of proposed attack and defense mechanisms, processing queries in real time in accordance with accuracy and latency constraints.
- A simple and query-efficient identification algorithm that enables practical black-box attacks on model-less inference serving systems.
- A differentially private defense strategy that prevents model identification using properties like accuracy and inference latency.
- Reduction in success of classic black-box attacks like extraction and membership inference using our `ParSec` defense.
- Configurable levels of defense, inducing a trade-off space between system utility and level of protection, providing the ability to achieve robust protection while maintaining high system performance.

Roadmap. §2 reviews key works on inference serving systems and black-box attacks. §3 introduces our threat model, which includes the server model and the adversary model. §4 describes our identification attack, and §5 introduces `ParSec` defense and details its implementation as a shim layer. §6 compares our defense's effectiveness against our identification attack and classic black-box attacks like model extraction and membership inference. §7 concludes the paper, summarizing our main findings and contributions.

2 Background and Related Work

2.1 Inference Serving Systems

TensorFlow Serving [1] was one of the first dedicated ML serving systems, although it was limited to models in the TensorFlow framework. Clipper [22] was later developed to use general frameworks and make it modular for anyone to deploy a model. Amazon SageMaker [35] and NVIDIA Triton Inference Serving [3] were some of the first publicly released serving systems officially offering inference as a service to satisfy business use cases. These systems have the advantage of large infrastructures backed by Amazon and NVIDIA, as SageMaker autoscales based on the inference load, and Triton optimizes inference on NVIDIA GPUs.

All of these systems require the user to specify the model used for inference, which may not satisfy all use cases. InferLine [21] provides serving for a pipeline of models by planning resource allocation for each model and tuning as necessary during execution. A prominent problem is a lack of developer understanding of the trade-off of accuracy/latency among variants of a model, such as the ResNet family [33]. Therefore, instead of having the developer specify the model to query, model-less systems that query from model zoos have arisen. INFaaS [62] generates variants for every model deployed to its zoo during the profiling process, and it navigates the trade-off space of these variants on the user's behalf. The clients simply provide the latency and accuracy bounds with their inputs in INFaaS.

However, many of these systems do not take advantage of the predictability of inference latency for a model, *i.e.*, deterministic forward pass latency of a deep neural network. Only recently has this attribute been studied, by systems such as Clockwork [29] and iGniter [74]. Clockwork achieves predictability by discarding queries that take too long, as well as by reducing the choice of resource allocation and program execution at the hardware, OS, and application level in order to reduce variability in end-to-end latency. This determinism allows for a model zoo to be represented as a Pareto frontier when plotted by its latency and accuracy, exhibiting the positive correlation between these two attributes in ML models. However, both of these systems require clients to specify the models they want along with the input.

Secure inference serving systems have been proposed to protect against malicious clients [14, 46, 65]. These works employ secure cryptographic protocols to protect against attacks that try to steal private inputs and parameters of the model. We consider an adversary (§3.2) that is only interested in extracting the functionality of a model and not its weights or training data.

Model Zoo. Models that are pre-trained for a specific task like image recognition can be coalesced into a repository known as a model zoo. Real-world examples have demonstrated the wide range of model zoo scales, with some containing as

few as five models [22], while others contain hundreds [31, 32]. Clockwork [29] uses the ONNX [2] and GluonCV [30] model zoos to keep 61 different models in its model zoo. INFaaS [62] contains 22 different architectures of models in its model zoo, each having a number of associated model variants combining for a total of 175 model variants. Each model variant is trained with different frameworks, compilers, batch sizes, and hardware platforms.

Developer APIs are used to interact with the model zoo of a system. For instance, the INFaaS [62] model-less interface has a declarative API that uses `register_model` to allow users to register models to its model zoo. To submit inference queries, developers use the `inference_query` API to specify high-level application requirements, such as accuracy and latency goals, without specifying the models. INFaaS then navigates the model zoo, selecting suitable model variants to meet the specified goals.

2.2 Black-box Attacks

Black-box attacks are attacks where the adversary lacks knowledge of the victim model's parameters, architecture, or training data. Machine Learning as a Service (MLaaS) are systems on which such attacks are usually carried out. In every query, the user typically submits an input and receives either a prediction vector or a class label from an already trained model hosted in the cloud. Most of these attacks are carried out during inference and, thus, on inference serving systems. Such attacks aim to obtain information not meant to be disclosed, such as the training data or details about the model. More information about black-box attacks can be found in §B.

3 Threat Model and Motivation

3.1 Server Model

We consider an inference serving scenario where models are loaded in the system like in Clockwork [29] or Ray [51], and it is the responsibility of the inference serving system to select a model for each query that satisfies the accuracy and latency specified by the query, like in INFaaS [62]. Predictability in inference serving systems is extremely important, and for that reason, state-of-the-art inference serving frameworks support multi-tenancy in a non-interfering manner by mapping their query traffic flow to different model-serving workers. In addition to exclusive access to GPU workers, applications are also typically mapped to different queues in the router. Clockwork maintains a single queue per model served. The combination of dedicated queues inside the serving system as well as dedicated GPU workers leads to a multi-tenant system with highly predictable tail latency guarantees, leveraged by `ParSec`. The `ParSec` server under consideration is honest-but-curious.

3.1.1 Pareto Front and Feasibility Set

For our purposes, the model zoo will be used to provide possible model selections to serve when performing inference. If we plot each model in the model zoo as a point on a scatter plot, such that the x-axis is the model's inference latency and the y-axis is its accuracy, we can select a subset of points that are preferred to the other points. This subset constitutes a **Pareto front**. It is a subset of points such that no point in the subset is strictly better than any other point when plotted against its chosen attributes. In our context, it means that no model will have both a lower latency and a higher accuracy compared to any other model on the Pareto front. By definition, every model zoo has a Pareto front.

Definition 1. *(Pareto front) Let a_m and l_m be the accuracy and latency values of any model m in the model zoo. For any $(a_i,l_i),(a_j,l_j) \in \mathbb{R}^2$, the Pareto frontier $\mathcal{P}$ is:*

$$\mathcal{P} = \{(a_i,l_i) | \{(a_j,l_j) | (a_j > a_i) \wedge (l_j < l_i)\} = \phi\}, \quad (1)$$

where ϕ is the null set, $(a_i,l_i) \neq (a_j,l_j)$, and $i \in \{1,\ldots,n\}$ with n being the total number of models in the model zoo. Thus, $\mathcal{P} \subseteq \mathbb{R}^{n\times 2}$. Since it is preferable to minimize inference latency and maximize accuracy, the Pareto frontier will form at the top left of the region of points representing the model zoo. The Pareto frontier will serve as the backbone for our identification algorithm, as it provides the adversary a path of traversal across the model zoo.

Since interactive applications have different latency SLOs for different requests, models in the model zoo typically span a broad latency-accuracy tradeoff space. Inference serving systems typically expose and serve the Pareto front of this tradeoff space, such that the highest accuracy model is served for a given latency budget, and the lowest latency model is served for a given accuracy threshold. Hence, serving using the Pareto-front helps in meeting deadlines, thus increasing utility of the system.

The **feasibility set** of a query is the set of models that satisfy the latency and accuracy specifications of the query. It is a subset of the Pareto frontier of the model zoo. The manner in which the serving system makes a model selection from the feasibility set, such as aggregation, random selection, round-robin, lowest-cost, *etc.*, provided that the set is non-empty, is known as the system's policy. If the accuracy and latency specifications of a query are acc and lat, respectively, then the feasibility set is $\mathcal{F} = \{(a_i,l_i) \in \mathcal{P} | (a_i \geq acc) \wedge (l_i \leq lat)\}$.

3.1.2 Inference Serving

To demonstrate that single-model attacks against a victim model from a model zoo can be successful in the simplest inference serving scenario, we employ the following two policies:

Cross-hair interface: The model serving endpoint accepts inference queries with specified minimum accuracy require-

ment a_{req} and maximum latency l_{req} requirement. A model will be randomly selected from the feasibility set defined by a_{req} and l_{req}. While it may make more sense to select the most cost-effective model from the feasibility set instead of random selection, we do not profile the resource requirements of each model and hence do not use it in the model selection process. The model selection policy can be easily changed to select the most cost-effective model, however, if the resource consumption is known to the profiler. If the feasibility set is empty, an "infeasible set error" will be returned to the client. Please note that the feasibility set is a subset of the Pareto frontier of the model zoo. This means the system serves models exclusively from the Pareto frontier like in [63]. A default value of 0 is set if a_{req} is not provided.

Granularity and Boundary: There is a granularity for accuracy (acc_g) and a granularity for latency (lat_g) that the inference serving system keeps track of beyond which adjacent values are indistinguishable. acc_g is less than the minimum difference between the accuracy values of any two consecutive models on the Pareto frontier. Similarly, lat_g is less than the minimum difference between the latency values of any two consecutive models on the Pareto frontier. For instance, the inference serving system might only keep track of accuracy to 0.1% and latency to 1 millisecond. Additionally, there is an upper bound for latency l_{up} in the system.

3.2 Adversary Model

Adversary Goal: To extract the most accurate model from the model zoo, given a specific latency budget, with a reasonably small number of queries. The adversary does not know the accuracy or latency specifications needed to target this model. It is important to note that this is not the same as simply extracting the most accurate model from the model zoo, as that model may have an inference latency greater than the latency budget of the adversary. This makes identification necessary as we will see in §4.1. The latency budget (L) is associated with every query. The adversary selects L based on its intended deployment scenario for the extracted model. The latency budget can be thought of as the latency SLO of the application or task associated with the query, like image classification or language translation. The adversary will try its best not to violate this budget for as many queries as possible so that it does not incur additional costs per query.

Type of Attack: This is an end-user attack because the adversary disguises itself as any other client trying to query the inference serving system for getting predictions on its input, using the cross-hair interface. Since the models are hidden in a model zoo behind a model-less interface, neither does the adversary know the accuracy or latency values of any model in the model zoo nor does it know the number of models present in the model zoo.

The information available to the adversary are the accuracy and latency values of each query along with the prediction made by the served model. The (accuracy, latency) information can be seen as *auxiliary* information acquired by the adversary. This is crucial information that enables the attacker to learn more about the Pareto frontier of the model zoo with every query it sends to the system.

4 Attack on Inference Serving Systems

4.1 Identification Attack on the Pareto Front

Since applications operate on diverse operating latency budgets, and target deployment devices may have diverse resource constraints, the highest accuracy model may not always be the attacker's desired target. Given the latency constraint of the attacker, the highest possible accuracy model that satisfies this constraint is preferred. Indeed, extracting Pareto optimal points with *lower* accuracy/latency tradeoff may be more suitable for embedded devices or autonomous cyber-physical systems that operate on a tight sensory-actuator feedback control loop and are resilient to lower accuracy [36]. To target a given point of interest on the Pareto frontier requires an *property* identification mechanism. A naive attack without an identification mechanism is shown in §E.

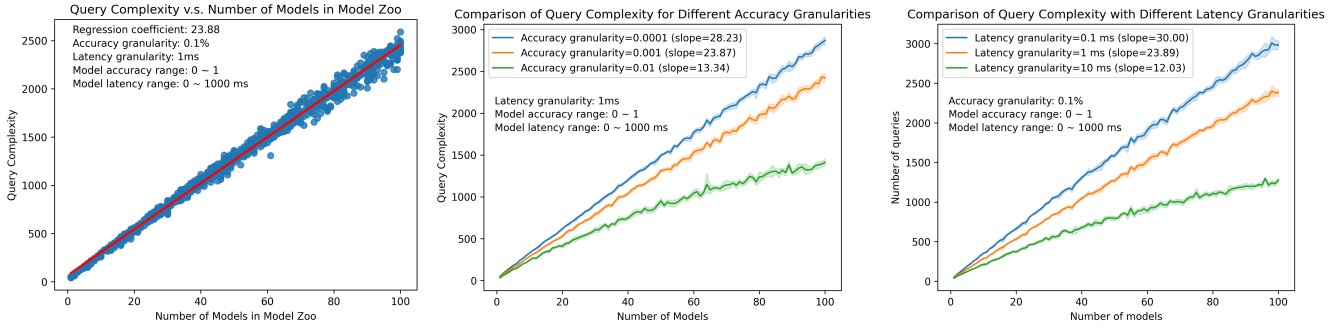
To make sure queries to the model zoo are routed to the victim model, we propose to fingerprint the Pareto front of the model zoo. Specifically, given black-box access to the model serving system, an adversary wants to find the accuracy and latency profile of every model on the Pareto frontier $\mathcal{P}$ (Eq. 1), and to leverage this information to send inference queries that consistently trigger the victim model in the model zoo.

4.1.1 Our Identification Algorithm

The challenge for fingerprinting is to use as few queries as possible. We propose a binary search-style algorithm. The intuition is that the system serves models exclusively from the Pareto frontier (§3.1). Due to the accuracy-latency correlation in the Pareto frontier, the search space is already sorted. So we use this path for traversing the search space in our algorithm.

First, given an inference serving endpoint, we perform a binary search to find the accuracy of the most accurate model on the Pareto frontier of the model zoo. To do this, we first send a query with $(a_{req}, l_{req}) = (0.5, \infty)$. If the query gets a response without error, it means there is at least one model in the model zoo with accuracy ≥ 0.5, and we send another query with $(a_{req}, l_{req}) = (0.75, \infty)$. On the other hand, if the query gets an error in response indicating that no model in the model zoo satisfies the requirement, we send another query with $(a_{req}, l_{req}) = (0.25, \infty)$. We stop the binary search process when we find a_{max} such that querying with $(a_{req}, l_{req}) = (a_{max}, \infty)$ gets a successful response and querying with $(a_{req}, l_{req}) = (a_{max} + a_g, \infty)$ gets an error.

Secondly, we use binary search to find the latency of the most accurate model on the Pareto frontier of the model zoo.



(a) The query complexity of the identification algorithm is $O(n)$.

(b) The effect of accuracy granularity on query complexity.

(c) The effect of latency granularity on query complexity.

Figure 2: Identification algorithm query complexity analysis. The slope can be interpreted as the number of queries an adversary has to expend per model in the model zoo. **Left:** We see that our identification algorithm takes $O(n)$ queries in the average case, where n is the number of models in the Pareto frontier of the model zoo. **Center and Right:** We see that the query complexity remains $O(n)$ for different values of accuracy granularity and latency granularity.

We first send a query with $(a_{req}, l_{req}) = (a_{max}, \frac{1}{2} \times l_{up})$. If the query gets a response without error, it means the most accurate model in the model zoo has latency greater than $\frac{1}{2} \times l_{up}$, and we send another query with $(a_{req}, l_{req}) = (a_{max}, \frac{3}{4} \times l_{up})$. On the other hand, if the query gets an error in response indicating that the most accurate model has latency less than $\frac{1}{2} \times l_{up}$, we send another query with $(a_{req}, l_{req}) = (a_{max}, \frac{1}{4} \times l_{up})$. We stop the binary search process when we find l_{max} such that querying with $(a_{req}, l_{req}) = (a_{max}, l_{max})$ gets a successful response and querying with $(a_{req}, l_{req}) = (a_{max}, l_{max} - l_g)$ gets an error.

Next, in a similar manner, we perform a binary search to find the accuracy and latency of the second most accurate model in the model zoo within the boundary $0 \leq a < a_{max}$ and $0 \leq l < l_{max}$. Then, we adjust the boundary and find the third most accurate model. The process continues till we find the accuracy and latency of all models in the model zoo. We return the result as a 2D matrix, $\mathcal{M}$, where the first column is $f_{acc}(\mathcal{P})$ and the second column is $f_{lat}(\mathcal{P})$. Algorithm 1 describes the identification algorithm.

Complexity Analysis. We measure the efficiency of the identification algorithm in terms of query complexity, *i.e.*, the total number of queries an adversary needs to expend to get the accuracy and latency of all models in the Pareto frontier. The worst case query complexity of the identification algorithm is $O(n(\log \frac{1}{a_g} + \log \frac{l_{up}}{l_g}))$, where n is the number of models in the Pareto frontier, giving linear time complexity $O(n)$. This is because we perform binary searches for both accuracy and latency for every single model in the model zoo. However, based on our experiments in which we simulated model zoos containing models with random accuracy and latency values, we found that the identification algorithm has a linear time average query complexity (see Fig. 2a). Further, Fig. 2b and Fig. 2c illustrate the effect on the linear regression coefficient by changing the accuracy or latency granularities.

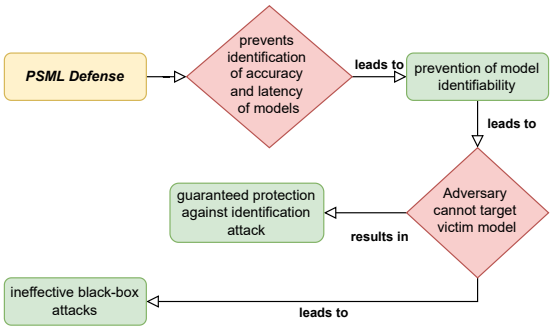


Figure 3: The Defense Workflow. `ParSec` defense helps is preventing property attacks, *i.e.*, identification of accuracy and latency values of models. This prevents the models from being identified using the identification attack. Now the adversary cannot target the victim model repeatedly. This not only gives us guaranteed protection against model identifiability, but also reduces the success of black-box attacks.

5 Defending Against Identification

5.1 Proposed Defense

A wide range of defenses against single-model extraction or membership inference has been proposed. One way is to perturb the output of models (*e.g.* [39,41,42,45,55,75]), but it is not a viable option for the system as a legitimate client also receives perturbed outputs. Watermarking (*e.g.* [6,38,67]) is another defense method against black-box attacks. It embeds a secret pattern in the model during inference or training, but it requires post hoc analysis and the model owner to have access to the extracted model. Another possible way is to detect malicious clients (*e.g.* [40,58]), but these methods have impractical assumptions like subsequent adversarial queries

Algorithm 1 Our Identification Algorithm. Accuracy is a discrete value between 0 and 1 and has a step size of acc_g. Similarly, latency is a discrete value ranging from 0 to a predetermined upper boundary, l_{up}, with an incremental step size of lat_g.

procedure IDENTIFICATION(l_{up})
  $\mathcal{M} \leftarrow [\,]$
  $acc_{up} \leftarrow 1$
  $lat_{up} \leftarrow l_{up}$
  while $acc_{up} \geq acc_g$ **do**
    $acc \leftarrow$ FIND_MAX_ACC(acc_{up}, lat_{up})
    $lat \leftarrow$ FIND_LAT(acc)
    if $acc \geq acc_g$ **then**
      $\mathcal{M}$.add((acc, lat))
    end if
    $acc_{up} \leftarrow acc - acc_g$
    $lat_{up} \leftarrow lat - lat_g$
  end while
  return $\mathcal{M}$
end procedure
procedure FIND_MAX_ACC(acc_{up}, lat_{up})
  $acc_{low} \leftarrow 0$
  $acc_{hi} \leftarrow acc_{up} + acc_g$
  while $acc_{hi} - acc_{low} \geq acc_g$ **do**
    $acc_{mid} \leftarrow (acc_{low} + acc_{hi})/2$
    $R \leftarrow$ infer(acc_{mid}, lat_{up})
    if R is error **then**
      $acc_{hi} \leftarrow acc_{mid}$
    else
      $acc_{low} \leftarrow acc_{mid} + acc_g$
    end if
  end while
  return $acc_{low} - acc_g$
end procedure
procedure FIND_LAT(acc)
  $lat_{low} \leftarrow 0$
  $lat_{hi} \leftarrow l_{up} + lat_g$
  while $lat_{hi} - lat_{low} \geq lat_g$ **do**
    $lat_{mid} \leftarrow (lat_{low} + lat_{hi})/2$
    $R \leftarrow$ infer(acc, lat_{mid})
    if R is error **then**
      $lat_{low} \leftarrow lat_{mid} + lat_g$
    else
      $lat_{hi} \leftarrow lat_{mid}$
    end if
  end while
  return $lat_{low} - lat_g$
end procedure

have small L^2 distances between them and that there are a mix of natural and synthetic queries that the adversary uses when trying to attack a model.

Since our primary goal is to stop single-model attacks on the model zoo, we concentrate on obscuring the Pareto front during inference serving so that the auxiliary information, *i.e.*the accuracy and latency specifications, is less useful to the attacker. Since the inference serving system only serves points from the Pareto front , we only want to protect the models on it, not those under it. To this end, we employ a differentially private mechanism that assigns a probability to every candidate model, *i.e.*, all models the Pareto front, based on a score function. If we use the system utility as the score function , then the mechanism will result in higher utility models having greater probability of being selected to serve the queries. This technique automatically maximizes utility as the probability of selecting the maximum utility model is the highest.

A defense should increase privacy while not reducing the system performance for the benign clients. Therefore, it is useful to measure the drop in system performance with increased privacy. We measure the system's performance using the **utility**. In our context, it is defined as the normalized L^1 distance of the selected model's accuracy and latency to the query's requested accuracy and latency specifications, respectively. With the DP defense, some models that are served will violate either the accuracy specification or latency specification, or both. This is because the defense allows any model from the Pareto front to be served, not just from the feasibility set . While the DP mechanism prevents model identification and decreases the success of black-box attacks, it inevitably reduces the utility of the inference serving system by inducing randomness in the model serving.

The idea of using DP to achieve privacy [24, 25] is used in diverse applications like inference [49, 71], feature extraction [56, 57], cloud [47], and systems [16, 19, 34, 72]. The novelty in our defense lies in the way we employ a DP mechanism to protect an inference-serving system, *i.e.*, by adding score-based randomization to the selection of models in a declarative model-less inference serving API .

5.2 `ParSec` Defense Algorithm

In order to protect the victim model in a model zoo, we must find a way to prevent the identification step from being successful. It is easy to see from Algorithm 1 that adding noise to the accuracy and latency values of models on the Pareto frontier would disrupt the identification attack . However, the accuracy of a pre-trained model in the zoo cannot be changed, and the inference latency can only be increased (by adding delay), not decreased.

Another possibility is to add noise to the profiled values (in the `ParSec`'s profiler) of accuracy and latency of each model in the `ParSec` server (Fig. 4). The identification step would now return noise-induced accuracy and latency values of the victim model. However, since we do not know when the adversary is fingerprinting and when it is collecting labeled examples for black-box attacks, the modified values would continue to be used by the model selection logic. Thus, the ad-

versary would still be able to target the victim model with the noise-induced accuracy and latency specifications it received from the identification step. Lastly, adding noise directly to the input query's accuracy and latency specifications will not allow us to satisfy any DP definition because to have a DP guarantee, the noise always has to be added to the output of the query, not the input.

First of all, the defense thwarts the model identification step, preventing the attacker from accurately determining the accuracy and latency values of its victim model in the model zoo. Secondly, it also makes the subsequent querying process uncertain due to the DP-induced randomness, adding extra protection to the system. It is easy to see that our defense strategy will work against any single-model black-box attack, not just model extraction or membership inference. However, for ease of experimentation (§6.3), we show the viability of our defense on only extraction and membership inference attacks.

Defense Formulation. We introduce two functions: $f^L_{acc}(\mathcal{P}) : \mathbb{R}^{n\times 2} \rightarrow \mathbb{R}$ and $f^L_{lat}(\mathcal{P}) : \mathbb{R}^{n\times 2} \rightarrow \mathbb{R}$, that return the accuracy and latency of the victim model, respectively, given the adversary's latency SLO (L). After the identification step, the adversary has *identified* the accuracy and latency values of all the models in the Pareto front. Next, it has to select the victim model based on these values and its latency SLO L, . In Algorithm 1, we fingerprint every model, regardless of the adversary's latency SLO, so that the adversary does not have to rerun the identification step on the model zoo again. Even if the adversary's latency SLO changes (increases or decreases) in the future, it can just pick the new victim model based on the values already obtained from the identification step.

The adversary picks row k in the Pareto front matrix $\mathcal{P}$, which has the largest latency value, l_k, below its latency budget L. Thus, $f^L_{lat}(\mathcal{P}) = l_k$. Then, it selects the accuracy value a_k from row k. Thus, $f^L_{acc}(\mathcal{P}) = a_k$. The obtained pair of values (a_k, l_k) serve as the adversary's accuracy and latency specifications of every subsequent query to the inference serving system. .

We use the differentially private *permute-and-flip mechanism* [48] that returns a candidate model with utility that is as high as possible for the given query. This uses a single parameter (ε) to quantify the probability of a candidate model to be selected for the query. The sensitivities quantify the maximal possible change of both accuracy and latency values, denoted by Δf^L_{acc} and Δf^L_{lat}, respectively. We prefer permute-and-flip to the commonly used exponential mechanism [26] because its expected utility is at least as big as the exponential mechanism. Results with the exponential mechanism are given in the appendix.

Composition and Privacy Amplification. In our identification attack, each query builds on top of the previous query. This results in privacy degradation as the number of queries increases, according to the composition theorems of DP [26]. To prevent the privacy from blowing up, several privacy amplification strategies have been proposed. One way is to divide the Pareto front into public and private subsets. Then the privacy, *i.e.*, ε only degrades if a model from the private subset has been selected [79]. Poisson subsampling is another way to amplify privacy that scales ε with a logarithmic factor every time a model from the private subset is selected [79]. We employ both these methods so that our privacy degradation is sub-linear with respect to the number of queries.

We discuss implementation details in §5.3. Algorithm 2 describes how we serve models with `ParSec` defense. The key idea is to iterate through the models in the Pareto front in random order and flip a coin to decide whether to select the model or not. The flip probability is higher for models with high utility.

Algorithm 2 Model Serving with Defense for a Query

```
1:  procedure SERVE_MODEL(P, L, ε, q)
2:      acc ← f^L_acc(P)
3:      lat ← f^L_lat(P)
4:      for m in RandomPermutation(P) do
5:          s_m ← utility(m, f^L_acc(P), f^L_lat(P))
6:          p_m = probability(Δf^L_acc, Δf^L_lat, s_m)
7:          if Bernoulli(p_m) then
8:              select m
9:          end if
10:     end for
11:     if m in PrivateSubset then
12:         ε' = DegradePrivacy(ε, q)
13:         q = q + 1
14:     end if
15:     return m, ε'
16: end procedure
```

5.3 System Design and Implementation

We build a model-less inference serving API as a shim layer that can be attached on top of real-world inference serving systems like Clockwork [29] and Ray [51], with plans to open source to the community. We used Ray for our experiments. Figure 4 illustrates our system design used to perform experiments with our proposed identification and defense mechanisms. Building upon the policies outlined in §3.1.2, we leverage a model server. Over its client library, we implemented a shim layer (`ParSec` Server) that exposes an inference API. This API allows clients to submit inference queries with specific minimum accuracy specification *acc*, maximum latency specification *lat*, and input to perform the inference. Thus, the entire system becomes model-less.

Upon loading models into the model zoo, the `ParSec` server maintains a table of the accuracy and latency values for each model. In the absence of any defense mechanism, upon receiving an inference query, the `ParSec` server selects the model

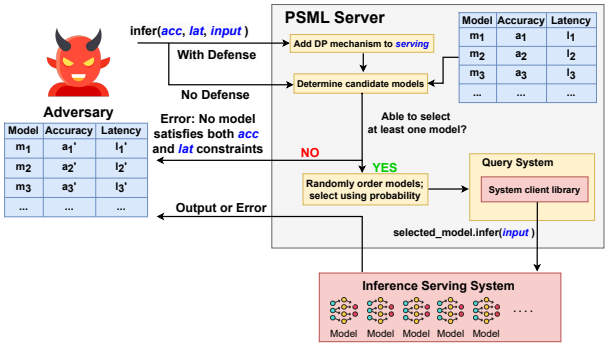


Figure 4: The system design and workflow for ParSec server. After the models are uploaded and profiled in the ParSec server, inference serving takes place through the interaction between the server process and the system process. The ParSec server routes the model architecture to be served to the system, and the system returns the output back to the server, which is then returned to the client.

from the feasibility set that minimizes the slack of the queue . The ParSec Server then forwards the inference query input to the system, which serves the query using the selected model, and relays the output from the system back to the client. An error is returned if no model satisfies both the accuracy and latency constraints (*i.e.*, the feasibility set is empty).

The shim layer includes the model selection policy that utilizes the accuracy and latency specifications of the client . In the inference serving system, the selected model information is relayed to the controller node, which subsequently assigns the inference task to the appropriate worker node , and the inference is performed on this model using the input provided by the client. Please note that the actual inference time of the query is not used by the adversary at all in our attack. With the defense mechanism active, the ParSec server introduces our DP mechanism before selecting the model to be served (Algorithm 2). There is a fundamental tradeoff between the privacy and utility of the system due to the chance that a sub-optimal model in terms of slackfit may be selected.

6 Evaluation

6.1 Experiment Setup

We evaluate our identification attack and our DP-based defense using the ParSec. The profiling of models are done using NVIDIA A40 GPUs. All experiments are run using 16 A40 GPUs. We employ SOTA model extraction (MixMatch [37], see §C.) and membership inference attacks (LiRA [11], see §D.) , on a zoo of image classification models trained on the CIFAR-10 and CIFAR-100 [44]. We show the following three results using our experiments:

1. black-box attacks can be successful using our identification algorithm
2. the identification algorithm can be prevented using our DP-based defense without compromising utility
3. black-box attacks can be weakened using our DP-based defense without compromising utility

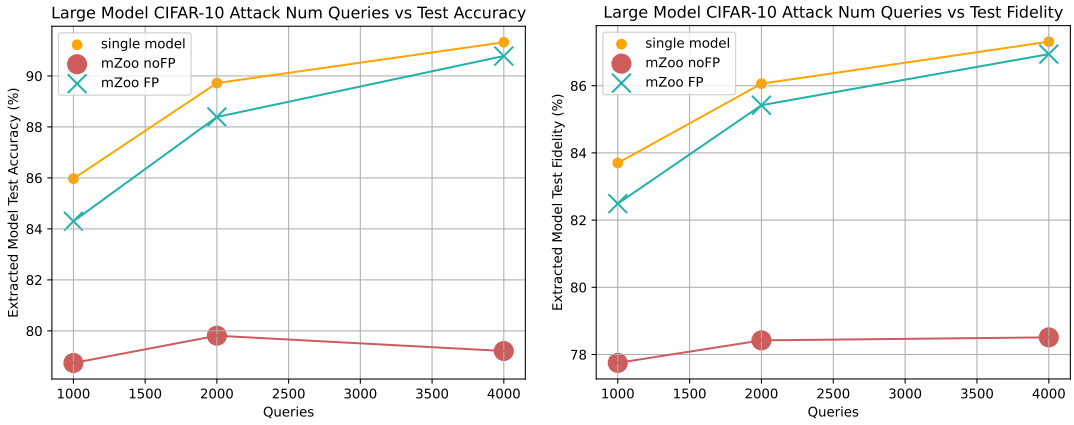
The extracted model (*i.e.*, the model that the attacker starts with) is a ResNet-50, which can reach $> 95\%$ accuracy on CIFAR-10 and $> 75\%$ accuracy on CIFAR-100. Thus, the model has enough expressive power to learn weights from the CIFAR-10 & CIFAR-100 train set. Based on the client's desired accuracy and latency specifications, the query is routed to a model that satisfies them. Otherwise, the system doesn't fulfill the query and sends an "infeasible set" error message to the client. It is important to note that our attack and defense have nothing to do with the training, architecture, or weights of the neural network models in the zoo. Therefore, our methods are plug-and-play, *i.e.*, no modification is needed on real-world model zoos that are usually dense and have models trained to their full potential on complex datasets like ImageNet [23].

The image classification models on the Pareto frontiers of our model zoos are comprised of **diverse** architectures of ResNets, WideResNets, DenseNets, MobileNets, ShuffleNets, VGGs, ViTs, and ConvNeXts. All of these models have predictable inference latency values. According to the definition of the Pareto frontier of a model zoo, models with larger inference latency have higher accuracy. We consider three querying scenarios with differing latency budgets available to the adversary: large, medium, and small latency budgets. Since with a larger latency budget, the adversary's victim model is a larger-sized model, we also call these scenarios the large, medium, and small model extraction cases. The query budget available to the adversary is 4000 queries in all cases. The inputs of these queries are randomly selected from the training set of the dataset. Hence, the adversary has to do the identification and querying within 4000 queries. We show how the adversary's success varies with different query budgets in Fig. 5.

6.2 Attack

We aim to demonstrate our identification algorithm's potential in adapting single-model attacks to inference serving systems. We show that our identification algorithm improves the success metrics of the black-box attack than without this step. It is important to note that the identification attack takes a constant number of queries for a given Pareto front, when acc_g and lat_g are fixed. This is because Algorithm 1 is deterministic.

We run attacks in each setting using a predetermined query budget. A query budget of q means that the attacker can query the system at most q times. This is also linked to privacy degradation as a larger number of queries means more privacy degradation. Thus, the attacker has to do the identification and get labeled points within these q queries. The attacker's



(a) Test Accuracy

(b) Test Fidelity

Figure 5: Accuracy and Fidelity of the extracted model for different query budgets in the large model setting. These experiments are done on CIFAR-10. We see that the fidelity and accuracy of the extracted model increase with increasing query budget of the adversary. However, in some cases, increasing the number of queries will not result in a better accuracy or fidelity score, as seen in the no-identification case (accuracy is higher with 2000 queries). This is because having more labeled points doesn't necessarily mean higher quality labeling in the no-identification case, as many of these points are labeled by very low accuracy models.

latency budget per query for the large-model setting is 21 ms, while it is 13 ms for the medium-model setting and 5 ms for the small-model setting. In each test, we let the MixMatch and LiRA attacks train for 512 epochs. Accuracy and fidelity of extraction and accuracy of membership inference of the victim model are measured against the test set.

6.2.1 Experiment Results

The extraction results are shown in Tab. 1 and Tab. 2 and the membership inference results are in Tab. 3. In the single-model attack, all 4000 queries were answered by the victim model. In the no-identification (mzoo naive) and identification (mzoo FP) attacks, all the queries were served using the `ParSec` server. A total of 411 queries were used for fingerprinting the model zoo. According to our threat model in §3.2, the attacker does not know the accuracy of the highest-accuracy model in the model zoo that has latency lower than its latency budget. On the other hand, the accuracy and latency values for all the models in the zoo were determined by the identification attack. Therefore, in the subsequent step of querying the system, the attacker simply chooses the accuracy and latency specifications corresponding to the model with the highest inference latency below its latency budget (see §4.1).

It is clear from the tables that the adversary can fingerprint the entire model zoo by expending a relatively low number of queries. This ability enables it to determine precise accuracy and latency values of the victim model. The test fidelity, test accuracy and membership inference accuracy results show that a black-box attack on a model zoo can be as successful as an attack on a single model. We can also see that the single-model attack is slightly better. This is because the number of queries used to fingerprint the model zoo is subtracted from the total query budget of the adversary. This means there are fewer labeled points from the victim model for the black-box attack. The main takeaway from these experiments is that an adversary can extract the highest accuracy (or largest) model with latency lower than a given latency budget from a model zoo using our identification algorithm.

Table 1: Accuracy values of the extracted model under different settings after using the MixMatch attack. The query budget is 4000. We see that the identification attack achieves accuracy close to that in the single-model setting for diverse model sizes. The defense successfully protects models from extraction with the given ε values. FP = fingerprinting (*i.e.*, our identification attack).

Setting	accuracy of extraction			
	CIFAR-10			CIFAR-100
	small	medium	large	small
single model	77.13	91.32	94.21	59.87
mzoo naive (§E)	62.38	78.23	83.57	50.78
mzoo FP	76.54	90.37	94.03	58.94
Defense ε=1	71.98	83.64	89.73	51.74
Defense ε=0.5	67.02	81.13	88.24	51.82
Defense ε=0.1	63.78	80.11	87.19	51.34

6.3 Defense

To demonstrate that our defense works on an actual inference serving system, we evaluate `ParSec` defense mechanism on our inference serving system. The goal of the defense is to undermine the identification step. By introducing randomness in the model selection , we hope to reduce the accuracy

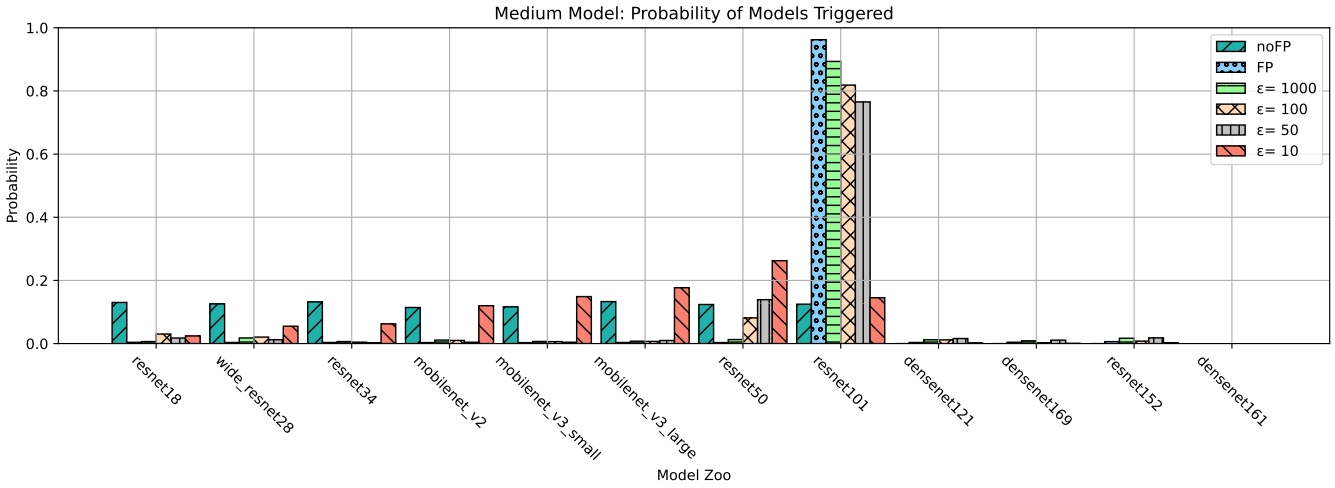


Figure 6: Probability mass functions plotted over the model zoo with different styles representing different attack/defense scenarios. ResNet-101 is the victim model. The plot shows: (a) ineffective extraction attack of the victim model without identification (`noFP`); (b) effective attack with high probability of calling the target model with proposed identification attack (`FP`); (c) mitigating the efficacy of the proposed attack with `ParSec` defense, dissipating the PMF over the base.

Table 2: Fidelity values of the extracted model under different settings after using the MixMatch attack. The query budget is 4000. We see that the identification attack achieves fidelity close to that in the single-model setting for diverse model sizes. The defense successfully protects models from extraction with the given ε values. FP = fingerprinting (*i.e.*, our identification attack).

Setting	fidelity of extraction		
	CIFAR-10		
	small	medium	large
single model	66.28	79.95	90.24
mzoo naive (§E)	59.12	71.71	78.31
mzoo FP	66.18	79.31	89.93
Defense ε=1	60.29	73.94	81.63
Defense ε=0.5	59.92	71.41	80.92
Defense ε=0.1	59.83	71.28	80.07

Table 3: Accuracy values of membership inference under different settings after using the LiRA attack. The query budget is 4000. We see that the identification attack achieves accuracy close to that in the single-model setting. The defense successfully protects models from membership inference with the given ε values.

Setting	accuracy	
	CIFAR-10	CIFAR-100
	small	small
single model	74.63	67.63
mzoo naive (§E)	62.79	61.92
mzoo FP	73.93	67.11
Defense ε=1	71.21	66.19
Defense ε=0.5	67.33	64.56
Defense ε=0.1	64.31	60.12

of the identification step. At the same time, we do not want to destroy the utility of the inference serving system. The permute-and-flip mechanism described in §5 is applied to the identification algorithm that is treated as a function. Since any sort of protection will be at odds with the system's performance, we hope to show the trade-off between them in this section through our experiment results.

We conduct our defense experiments on the same settings under which the attack is tested. Since the identification step is vital for the attack to succeed, we aim to see to what extent our defense reduces the effectiveness of this step. Ideally, we would like to reduce the success of black-box attacks to the values obtained when they were run without the identification step (Tab. 1, Tab. 2, and Tab. 3). The rationale behind adding DP based randomness in selection is that by changing the model that the inference serving system has to pick, we reduce the chances of successfully identifying any model from the zoo. As an intermediate step, we measure how well the defense does in preventing the identification attack in Tab. 5, where we also show that our `ParSec` defense helps in maintaining utility around 90% without blowing up the privacy budget, *i.e.*, the end ε value.

The performance of the inference serving system is measured in terms of utility, defined in Section 5.1. The system's goal is to be as performant as possible while maintaining a required degree of protection. Our defense mechanism prevents identification, but cannot change the quality of the pre-trained models on the Pareto frontier.

In the experiments, we set the system with sensitivity values $\Delta f_{acc} = 100$ and $\Delta f_{lat} = 100$ ms for CIFAR-10. We demonstrate extracting a large, medium, and small model. In these three scenarios, the latency SLOs of the attacker are 21 ms, 13 ms, and 5 ms, respectively, for CIFAR-10. Our experiments

Table 4: Mean number of queries used in each step for medium model extraction on a different model zoo. Query budget is 4000. Identification without defense always takes a constant number of queries. In the defense, query complexity of identification reduces with decreasing ε. We also show the number of queries that pass/fail.

Setting	number of queries			
	identification	labeling	successful	failed
single model	0	4000	4000	0
mzoo naive (§E)	0	4000	4000	0
mzoo FP	411	3589	3589	0
Defense ε=1000	901.33	3098.67	3093.33	5.33
Defense ε=100	864.33	3135.67	2310.00	825.67
Defense ε=50	867.33	3132.67	2007.00	1125.67
Defense ε=10	406.00	3594.00	3075.33	518.67

verify whether launching the black-box single-model attack with identification on a system that has our defense shows performance comparable to that in the no-identification (mzoo naive) case (*i.e.*, a naive attacker using single-model attacks on a model zoo without fingerprinting the zoo first).

In the defense, the number of queries taken to do identification is not constant for a given model zoo because of the randomness in the system. Moreover, not all queries will be answered by the system. A breakdown of the number of queries used in each step is presented in Tab. 4. Since the number of queries taken for defense is not constant, we did three trials for each setting and reported the average. After querying, the MixMatch method is trained for 1024 epochs on the points in the training set. Fig. 6 shows the probability distribution of the models getting triggered in different attack scenarios on another model zoo.

6.3.1 Experiment Results

We observe that the attacks are less successful with the defense in place. The closer the values of the success metrics is to those in the no-identification setting, the better the protection against the black-box attack. From Tab. 1 and Tab. 2, we see that the fidelity and accuracy scores decrease with decreasing epsilon values. A similar trend is seen with accuracy in Tab. 3. This is because lower epsilon results in more randomness. And more randomness in the system results in more queries being routed to models that are not the victim model. The results in Tab. 5 show that the success of the identification attack has reduced a lot with the defense, when we run the attack for the same number of queries.

Since higher protection comes with lower performance, we calculated the utility scores of these runs. We observe that utility decreases with decreasing epsilon values. The reason behind this is the fact that a lower epsilon value means more randomness. Utility around 90% is considered very good for an inference serving system. From Tab. 5, we can see that utility is around 90% for all of the runs, while the success of the corresponding black-box attacks reduce significantly.

Table 5: Hit rate for model zoos of different sizes with $\varepsilon = 1$. Here we see that with `ParSec` defense, the hit rate reduces as compared to without `ParSec` defense. The utility is around 90% and the end ε doesn't blow up due to the amplification and composition mechanisms in place.

num models	queries	w/o def	w/ def	utility	end ε
18	389	100	22.22	88.74	1.71
20	411	100	25	88.94	2.75
30	502	100	20	88.31	3.18
50	750	100	22	88.87	4.82

Hence, the system designer can use these ε values to effectively defend against black-box attacks while maintaining an acceptable quality-of-service (QoS). The main takeaway is that a balance between protection and performance can be reached by configuring ε in our defense mechanism.

We see that the defense is more effective in the medium and small model settings than in the large model setting. This is because while extracting a large model, even a large amount of randomness in the system may result in a high-accuracy model being served. Since high-accuracy models result in high-quality labeled examples, the black-box attack will not suffer much, and the resulting accuracy values will remain fairly high. We provide a lower bound on fidelity while extracting a large model in Appendix A. Thus, our defense mechanism with the tested ε values is more effective for attacks targeting medium to small models. However, ε can be configured to suit the system designer's needs.

7 Conclusion

We observe that traditional black-box attacks rely on outdated assumptions that the victim model can be explicitly specified and queried. This is no longer true in second generation inference serving systems. Thus, naively adapting single-model black-box attacks on such systems is not effective. Our novel, query-efficient identification algorithm re-enables black-box attacks on such systems. We defend against the proposed identification step with `ParSec`, a novel defense mechanism based on ε−differential privacy. Doing so helps expose a practical tradeoff space between the system's level of privacy and its performance, captured by utility. We show that robust levels of privacy can be achieved without compromising system performance. The proposed identification step and defense mechanisms can be seamlessly integrated with a real system. With a growing number of proprietary models served via model selection in second generation inference serving systems, we make the first step towards understanding the security implications of such systems with respect to Intellectual Property theft through black-box attacks. We mention some potential future works in §F.

References

[1] TensorFlow Serving for Model Deployment in Production, 2018. https://www.tensorflow.org/tfx/guide/serving.

[2] The ONNX Model Zoo. https://github.com/onnx/models, 2020.

[3] Nvidia Triton. https://developer.nvidia.com/nvidia-triton-inference-server, May 2023.

[4] Qualcomm Works with Meta to Enable On-device AI Applications Using Llama 2, July 2023.

[5] Llama-v2-7B-Chat. https://aihub.qualcomm.com/models/llama_v2_7b_chat_quantized, 2024.

[6] Yossi Adi, Carsten Baum, Moustapha Cisse, Benny Pinkas, and Joseph Keshet. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, pages 1615–1631, 2018.

[7] Giuseppe Ateniese, Luigi V Mancini, Angelo Spognardi, Antonio Villani, Domenico Vitali, and Giovanni Felici. Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers. *International Journal of Security and Networks*, 10(3):137–150, 2015.

[8] Payman Behnam, Jianming Tong, Alind Khare, Yangyu Chen, Yue Pan, Pranav Gadikar, Abhimanyu Bambhaniya, Tushar Krishna, and Alexey Tumanov. Subgraph stationary hardware-software inference co-design. In *Proceedings of the 6th Conference on Machine Learning and Systems*, MLSys'23, 2023.

[9] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. Mixmatch: A holistic approach to semi-supervised learning. *Advances in neural information processing systems*, 32, 2019.

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[11] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. Membership inference attacks from first principles. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1897–1914. IEEE, 2022.

[12] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic extraction of neural network models. In *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III*, pages 189–218. Springer, 2020.

[13] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX security symposium (USENIX security 19)*, pages 267–284, 2019.

[14] Nishanth Chandran, Divya Gupta, Sai Lakshmi Bhavana Obbattu, and Akash Shah. {SIMC}:{ML} inference secure against malicious clients at {Semi-Honest} cost. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1361–1378, 2022.

[15] Varun Chandrasekaran, Kamalika Chaudhuri, Irene Giacomelli, Somesh Jha, and Songbai Yan. Exploring connections between active learning and model extraction. In *Proceedings of the 29th USENIX Conference on Security Symposium*, pages 1309–1326, 2020.

[16] Chien-Ying Chen, Debopam Sanyal, and Sibin Mohan. Indistinguishability prevents scheduler side channels in real-time systems. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 666–684, 2021.

[17] Hyunji Chung, Michaela Iorga, Jeffrey Voas, and Sangjin Lee. Alexa, can i trust you? *Computer*, 50(9):100–104, 2017.

[18] Jacson Rodrigues Correia-Silva, Rodrigo F Berriel, Claudine Badue, Alberto F de Souza, and Thiago Oliveira-Santos. Copycat cnn: Stealing knowledge by persuading confession with random non-labeled data. In *2018 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2018.

[19] Jorge Cortés, Geir E Dullerud, Shuo Han, Jerome Le Ny, Sayan Mitra, and George J Pappas. Differential privacy in control and network systems. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 4252–4272. IEEE, 2016.

[20] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, pages 191–198, 2016.

[21] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. Inferline: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of*

the 11th ACM Symposium on Cloud Computing, pages 477–491, 2020.

[22] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *NSDI*, volume 17, pages 613–627, 2017.

[23] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[24] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Advances in Cryptology-EUROCRYPT 2006: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28-June 1, 2006. Proceedings 25*, pages 486–503. Springer, 2006.

[25] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4-7, 2006. Proceedings 3*, pages 265–284. Springer, 2006.

[26] Cynthia Dwork, Aaron Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[27] Neil Zhenqiang Gong and Bin Liu. You are who you know and how you behave: Attribute inference attacks via users' social friends and behaviors. In *USENIX Security Symposium*, pages 979–995, 2016.

[28] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, November 2020.

[29] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving dnns like clockwork: Performance predictability from the bottom up. *arXiv preprint arXiv:2006.02464*, 2020.

[30] Jian Guo, He He, Tong He, Leonard Lausen, Mu Li, Haibin Lin, Xingjian Shi, Chenguang Wang, Junyuan Xie, Sheng Zha, et al. Gluoncv and gluonnlp: Deep learning in computer vision and natural language processing. *The Journal of Machine Learning Research*, 21(1):845–851, 2020.

[31] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, et al. The architectural implications of facebook's dnn-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 488–501. IEEE, 2020.

[32] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 620–629. IEEE, 2018.

[33] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 630–645. Springer, 2016.

[34] Zhenqi Huang, Yu Wang, Sayan Mitra, and Geir E Dullerud. On the cost of differential privacy in distributed control systems. In *Proceedings of the 3rd international conference on High confidence networked systems*, pages 105–114, 2014.

[35] Doug Hudgeon and Richard Nichol. Machine Learning for Business: Using Amazon Sagemaker and Jupyter. https://aws.amazon.com/sagemaker, 2020.

[36] David Isele, Reza Rahimi, Akansel Cosgun, Kaushik Subramanian, and Kikuo Fujimura. Navigating occluded intersections with autonomous vehicles using deep reinforcement learning. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 2034–2039. IEEE, 2018.

[37] Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. High accuracy and high fidelity extraction of neural networks. In *Proceedings of the 29th USENIX Conference on Security Symposium*, pages 1345–1362, 2020.

[38] Hengrui Jia, Christopher A Choquette-Choo, Varun Chandrasekaran, and Nicolas Papernot. Entangled watermarks as a defense against model extraction. In *USENIX Security Symposium*, pages 1937–1954, 2021.

[39] Jinyuan Jia, Ahmed Salem, Michael Backes, Yang Zhang, and Neil Zhenqiang Gong. Memguard: Defending against black-box membership inference attacks via

adversarial examples. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 259–274, 2019.

[40] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N Asokan. Prada: protecting against dnn model stealing attacks. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 512–527. IEEE, 2019.

[41] Sanjay Kariyappa, Atul Prakash, and Moinuddin K Qureshi. Protecting dnns from theft using an ensemble of diverse models. In *International Conference on Learning Representations*, 2021.

[42] Sanjay Kariyappa and Moinuddin K Qureshi. Defending against model stealing attacks with adaptive misinformation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2020.

[43] John S Koh, Jason Nieh, and Steven M Bellovin. Encrypted cloud photo storage using google photos. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, pages 136–149, 2021.

[44] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[45] Taesung Lee, Benjamin Edwards, Ian Molloy, and Dong Su. Defending against neural network model stealing attacks using deceptive perturbations. In *2019 IEEE Security and Privacy Workshops (SPW)*, pages 43–49. IEEE, 2019.

[46] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. Muse: Secure inference resilient to malicious clients. In *USENIX Security Symposium*, pages 2201–2218, 2021.

[47] Sam Leroux, Tim Verbelen, Pieter Simoens, and Bart Dhoedt. Privacy aware offloading of deep neural networks. *arXiv preprint arXiv:1805.12024*, 2018.

[48] Ryan McKenna and Daniel R Sheldon. Permute-and-flip: A new mechanism for differentially private selection. *Advances in Neural Information Processing Systems*, 33:193–203, 2020.

[49] Fatemehsadat Mireshghallah, Mohammadkazem Taram, Prakash Ramrakhyani, Ali Jalali, Dean Tullsen, and Hadi Esmaeilzadeh. Shredder: Learning noise distributions to protect inference privacy. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 3–18, 2020.

[50] Abhishek Mishra. *Machine learning in the AWS cloud: Add intelligence to applications with Amazon Sagemaker and Amazon Rekognition*. John Wiley & Sons, 2019.

[51] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, pages 561–577, 2018.

[52] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011.

[53] Jerzy Neyman and Egon Sharpe Pearson. Ix. on the problem of the most efficient tests of statistical hypotheses. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, 231(694-706):289–337, 1933.

[54] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4954–4963, 2019.

[55] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Prediction poisoning: Towards defenses against dnn model stealing attacks. *arXiv preprint arXiv:1906.10908*, 2019.

[56] Seyed Ali Osia, Ali Shahin Shamsabadi, Sina Sajadmanesh, Ali Taheri, Kleomenis Katevas, Hamid R Rabiee, Nicholas D Lane, and Hamed Haddadi. A hybrid deep learning architecture for privacy-preserving mobile analytics. *IEEE Internet of Things Journal*, 7(5):4505–4518, 2020.

[57] Seyed Ali Osia, Ali Taheri, Ali Shahin Shamsabadi, Kleomenis Katevas, Hamed Haddadi, and Hamid R Rabiee. Deep private-feature extraction. *IEEE Transactions on Knowledge and Data Engineering*, 32(1):54–66, 2018.

[58] Soham Pal, Yash Gupta, Aditya Kanade, and Shirish Shevade. Stateful detection of model extraction attacks. *arXiv preprint arXiv:2107.05166*, 2021.

[59] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pages 506–519, 2017.

[60] Adnan Siraj Rakin, Md Hafizul Islam Chowdhuryy, Fan Yao, and Deliang Fan. Deepsteal: Advanced model extractions leveraging efficient weight stealing in memories. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1157–1174. IEEE, 2022.

[61] Robert Nikolai Reith, Thomas Schneider, and Oleksandr Tkachenko. Efficiently stealing your machine learning models. In *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society*, pages 198–210, 2019.

[62] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. Infaas: Automated model-less inference serving. In *USENIX Annual Technical Conference*, pages 397–411, 2021.

[63] Manas Sahni, Shreya Varshini, Alind Khare, and Alexey Tumanov. Compofa: Compound once-for-all networks for faster multi-platform deployment. *arXiv preprint arXiv:2104.12642*, 2021.

[64] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2017.

[65] Wenting Zheng Srinivasan, PMRL Akshayaram, and Popa Raluca Ada. Delphi: A cryptographic inference service for neural networks. In *Proc. 29th USENIX Secur. Symp*, pages 2505–2522, 2019.

[66] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.

[67] Sebastian Szyller, Buse Gul Atli, Samuel Marchal, and N Asokan. Dawn: Dynamic adversarial watermarking of neural networks. In *Proceedings of the 29th ACM International Conference on Multimedia*, pages 4417–4425, 2021.

[68] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[69] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction apis. In *USENIX security symposium*, volume 16, pages 601–618, 2016.

[70] Jean-Baptiste Truong, Pratyush Maini, Robert J Walls, and Nicolas Papernot. Data-free model extraction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4771–4780, 2021.

[71] Ji Wang, Jianguo Zhang, Weidong Bao, Xiaomin Zhu, Bokai Cao, and Philip S Yu. Not just privacy: Improving performance of private deep learning in mobile cloud. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2407–2416, 2018.

[72] Yu Wang, Zhenqi Huang, Sayan Mitra, and Geir E Dullerud. Differential privacy in linear distributed control systems: Entropy minimizing mechanisms and performance tradeoffs. *IEEE Transactions on Control of Network Systems*, 4(1):118–130, 2017.

[73] Carole-Jean Wu, David Brooks, Kevin Chen, Douglas Chen, Sy Choudhury, Marat Dukhan, Kim Hazelwood, Eldad Isaac, Yangqing Jia, Bill Jia, et al. Machine learning at facebook: Understanding inference at the edge. In *2019 IEEE international symposium on high performance computer architecture (HPCA)*, pages 331–344. IEEE, 2019.

[74] Fei Xu, Jianian Xu, Jiabin Chen, Li Chen, Ruitao Shang, Zhi Zhou, and F. Liu. igniter: Interference-aware gpu resource provisioning for predictable dnn inference in the cloud. *IEEE Transactions on Parallel and Distributed Systems*, 34:812–827, 2022.

[75] Mingfu Xue, Chengxiang Yuan, Can He, Yinghao Wu, Zhiyu Wu, Yushu Zhang, Zhe Liu, and Weiqiang Liu. Use the spear as a shield: an adversarial example based privacy-preserving technique against membership inference attacks. *IEEE Transactions on Emerging Topics in Computing*, 11(1):153–169, 2022.

[76] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)*, pages 268–282. IEEE, 2018.

[77] Jeong-Min Yun, Yuxiong He, Sameh Elnikety, and Shaolei Ren. Optimal aggregation policy for reducing tail latency of web search. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 63–72, 2015.

[78] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412*, 2017.

[79] Yuqing Zhu and Yu-Xiang Wang. Poission subsampled rényi differential privacy. In *International Conference on Machine Learning*, pages 7634–7642. PMLR, 2019.

A Proof of Lower Bound on Fidelity for Large Models

How is the fidelity of the extracted model with respect to the victim model related to their accuracy scores on the same test set?

To justify the defense, we need to show that it reduces the fidelity and accuracy scores. Extraction with randomness in the system will result in an "aggregate" extracted model. Thus, our goal is to show that this aggregate extracted model is poor in terms of fidelity and accuracy, i.e., it has a low fidelity score with respect to the victim model. It is safe to assume that a high-accuracy feasibility set will result in a high-accuracy extracted model. So now we will try to **minimize** the fidelity score between an aggregate extracted model and a victim model.

Let us calculate the minimum fidelity possible for an aggregate extracted model (i.e., obtained with the defense) from a high-accuracy feasibility set.

Proof. Let D be the test set of points for a classification task with k classes. Let S_i be the set of points from D that model m_i classifies correctly. There are two models of concern here: m_v (the victim model) and m_e (the extracted model).

Let a_v and a_e be the accuracy scores on D of models m_v and m_e, respectively. We assume that $a_v \geq 0.9$ and $a_e \geq 0.9$, as the extracted model m_e is obtained from querying a high-accuracy feasibility set and the victim model m_v lies in this feasibility set.

Let $n(A)$ stand for the number of distinct elements in the set A. Let F be the fidelity set, i.e., the set of all such points in D for which m_v and m_e predict the same class. F can be broken down into two disjoint sets:

1. a set of points on which both models make the correct prediction, i.e., $(S_v \cap S_e)$.

2. a set of points on which both models make incorrect predictions but predict the same class. We denote this set by P.

Thus, $F = (S_v \cap S_e) + P$ and $n(F) = n(S_v \cap S_e) + n(P)$

$n(S_v \cap S_e) = n(S_v) + n(S_e) - n(S_v \cup S_e) = a_v.n(D) + a_e.n(D) - n(S_v \cup S_e)$

Hence, $n(F) = a_v.n(D) + a_e.n(D) - n(S_v \cup S_e) + n(P)$

We can minimize the above as follows:
$min(n(F)) = a_v.n(D) + a_e.n(D) - max(n(S_v \cup S_e)) + min(n(P)) = a_v.n(D) + a_e.n(D) - n(D) + 0 = n(D).(a_v + a_e - 1)$

Thus, the minimum fidelity score is $\frac{min(n(F))}{n(D)} = (a_v + a_e - 1) = 0.8$ ■

Conclusion: Attacking a high accuracy (or large) model results in a relatively high fidelity score (≥ 0.8) even with the defense. Therefore, we need to attack a lower accuracy (or small) model to get a significant reduction in fidelity (or improvement in protection) with the defense.

B More on Black-box Attacks

The adversary's goal in model extraction is to replicate the functionality of the victim model by creating an extracted model [12, 15, 18, 37, 40, 54, 59–61, 69, 70]. It leverages the ability to query the victim model and observe its outputs, which are utilized to train the extracted model. Task accuracy attacks involve creating a model that matches the victim model's accuracy on a test set derived from the input data distribution. Fidelity attacks, however, aim to maximize the similarity between the victim and extracted models on the test set. Fidelity can be defined as the ratio of points in the test set on which both the victim and the extracted models have the same output labels. Attackers with problem domain data require fewer queries, and access to output labels alone is adequate for them to extract a model. In both scenarios, the adversary aims for efficiency, striving to minimize the number of queries used.

In membership inference attacks [11, 13, 64, 76], the adversary's goal is to determine whether a specific data sample was part of the training set used to train a target model. This type of attack exploits differences in how models behave when presented with data they have seen during training versus unseen data. For example, models may exhibit higher confidence or distinctive patterns in their predictions for training data compared to non-training data, which the adversary seeks to exploit. The broader goal of the adversary can vary depending on the context. In some cases, the adversary aims to extract private or sensitive information about individuals represented in the training set, posing significant privacy risks. This is particularly concerning in scenarios involving sensitive domains like healthcare, finance, or user behavior.

These attacks assume that the outputs received by the adversary are all from the victim model; hence, we also call them single-model attacks in this work. The attack described in [46] is on inference serving systems, but it assumes that the adversary can access the victim model repeatedly from the model zoo.

C MixMatch Model Extraction

MixMatch [9] is a semi-supervised learning method that guesses low-entropy labels for data-augmented unlabeled examples and mixes labeled and unlabeled data using

MixUp [78]. Given a batch of labeled examples with one-hot targets and an equally sized batch of unlabeled examples, MixMatch produces a processed batch of augmented labeled examples and a batch of augmented unlabeled examples with "guessed" labels. These two batches are then used in computing separate labeled and unlabeled loss terms.

Data augmentation is used on both labeled and unlabeled data. For each point in the batch of labeled data, an augmented version is generated. For each point in the batch of unlabeled data, K augmentations are generated. These individual augmentations are used to generate a "guessed" label using a label-guessing process. Both labeled examples and unlabeled examples with label guesses are used in MixUp. The loss function combines cross-entropy loss between labels and model predictions from the batch of augmented labeled examples with squared L_2 loss between guessed labels and model predictions from the batch of augmented unlabeled examples.

Jagielski *et al.* [37] leverage MixMatch as an extraction technique on the SVHN [52] and CIFAR-10 [44] datasets. For both datasets, inputs are color images of 32×32 pixels belonging to one out of ten classes. The victim model is a WideResNet-28-2 architecture that achieves 97.36% and 95.75% accuracy on SVHN and CIFAR-10, respectively. The adversary is given access to the same training set but without knowledge of the labels. The results of the MixMatch attack show that the adversary needs to query the victim model on a small subset of the training points to extract a model whose accuracy on the task is comparable to the victim model's. MixMatch is able to exploit a few labels because of the prior it is able to build using the unlabeled data. This results in improved test set accuracy and fidelity.

D LiRA Membership Inference

The LiRA attack [11] is a membership inference attack used evaluate privacy vulnerabilities in machine learning models. It refines the attack framework by focusing on the statistical behavior of models under hypothesis testing. LiRA computes the likelihood ratio between two hypotheses: the instance being a member of the training set versus it being from the same data distribution but not part of the training data. This approach captures varied signals in the model's outputs, such as confidence scores. This enables the attack to infer membership status with higher accuracy. By leveraging principles of Neyman-Pearson hypothesis testing [53], LiRA achieves state-of-the-art results across various datasets and architectures.

A critical insight of LiRA is its reliance on model generalization and overfitting behaviors. This helps to exploit how models treat training data differently from unseen data. Unlike earlier heuristic methods, LiRA trains shadow models to approximate the behavior of the target model and uses it to estimate membership likelihoods. This provides a robust way to measure privacy leakage without needing access to the target model's parameters. LiRA can succeed even when access is restricted to only model outputs. The attack is evaluated on standard datasets like CIFAR-10, CIFAR-100, and Purchase-100. It consistently outperforms existing membership inference attacks in terms of both accuracy and precision. The attack's performance is even better in overfitted models, where it can effectively leverage overconfidence in predictions for training data.

E Example of a Naive Attack

Table 6: Accuracy and Fidelity scores of MixMatch model extraction in the single-model and the model zoo setting. Here, the victim model is DenseNet-161 and the extracted model is ResNet-50. The adversary naively tries to adapt single-model attacks to the model zoo setting, *i.e.*, without identification.

Setting	Accuracy	Fidelity
single model	92.00	87.13
model zoo	81.04	79.84

The MixMatch attack can be used without any modification to extract a model from a model zoo as per the adversary goal. The client has to simply specify its latency budget to the inference serving system. Since it does not know the accuracy of its victim model, it does not provide an accuracy specification (*i.e.*, the default value 0 is selected). We conduct the MixMatch attack on a model zoo trained on the CIFAR-10 dataset [44] and separately run experiments for the single-model setting as well, where the only model is the victim model. Tab. 6 shows the accuracy and fidelity values obtained after training MixMatch for 1024 epochs. It is clear that the attack is significantly weaker in the model zoo setting as compared to the single-model setting. The model zoo can function as a layer of protection for the victim model, which means that naively using black-box single-model attacks will not be very successful on a model zoo unless there is a definitive way to route the queries to the victim model.

F Future Work

If DP is satisfied, any black-box attack (not just extraction and membership inference) that leverages the difference in the obtained accuracy or latency distributions for two neighboring tables of profiled latency and accuracy values of models will fail. As part of future work, it will be good to show that the obtained distributions without defense for two neighboring model profile tables are very different, but they become similar with our `ParSec` defense. Another interesting avenue of research is tying the effectiveness of our defense with the density or sparsity of the model zoo. A sparse model zoo

is not only ineffective for diverse deployment, but also will result in a weak defense, while a dense model zoo will result in a strong defense.